Pokhara University

Gandaki College of Engineering and Science

Lamachaur, Kaski

Lab 1: Implementation of Source Code Management Using Git Commands Through Github

Submitted By:

Abhimanyu Adhikari

Roll no. 04

Submitted To:

Er. Rajendra Bahadur Thapa

OBJECTIVE:

This lab applies Source Code Management practices through Git commands combined with GitHub.

AIM

To gain a working understanding of Source Code Management (SCM) concepts and apply them practically using Git together with GitHub, covering change tracking, version management, and collaborative development workflows.

THEORY

Source Code Management (SCM) refers to the practice of monitoring and controlling modifications made to a program's source files. It allows development teams to collaborate effectively, preserve a history of versions, and roll back to earlier states of the code when required.

Git functions as a distributed version control system, allowing each developer to record snapshots of file changes locally while coordinating contributions with teammates.

GitHub, in turn, is a web-based platform for hosting Git repositories, offering additional collaboration features such as pull requests, issue tracking, and project boards.

Together, Git and GitHub have become foundational tools in Agile software development, since they support teamwork, continuous integration, change tracking, and structured code review.

OBJECTIVES

- Understand the fundamental concepts behind Git.
- Set up and maintain Git repositories.

- Record project history through commits.
- Link a local repository to a remote GitHub repository.
- Synchronize project updates by pushing and pulling changes.
- Gain insight into how teams collaborate on software projects using version control.

ALGORITHM

1. Install Git on the local machine.
2. Set up an account on GitHub.
3. Create a new repository within GitHub.
4. Initialize a Git repository in the local project folder.
5. Add the project's files to the repository.
6. Stage the added files using Git.
7. Commit the staged changes.
8. Connect the local repository to its GitHub counterpart.
9. Push the committed files to GitHub.
10. Confirm that synchronization was successful.

CODE

- Initializing the Repository

```
git init
```

- Setting Up User Configuration

```
git config --global user.name "your name"
git config --global user.email "your@email.com"
```

- Checking Repository Status

```
git status
```

- Staging Files

```
git add .
```

- Committing Changes

```
git commit -m "Initial Commit"
```

- Linking the Repository to GitHub

```
git remote add origin https://github.com/username/repository.git
```

- Pushing to GitHub

```
git branch -M main
git push -u origin main
```

- Pulling the Latest Changes

```
git pull origin main
```

OUTPUT

Following execution of the above commands, the repository was successfully set up and linked with GitHub.

- A repository was created without errors.
- Project files were placed under Git's version tracking.
- Modifications were committed successfully.
- A remote GitHub repository was linked.
- The project was pushed to the GitHub remote.

RESULT

The exercise in Source Code Management using Git and GitHub was carried out successfully. Core version control tasks, including repository creation, staging, committing, and synchronization with GitHub, were all executed without issue.

CONCLUSION

This lab exercise offered hands-on exposure to Source Code Management with Git and GitHub. It highlighted how version control systems support efficient project management by preserving code history, enabling collaborative work, and reinforcing key practices used in Agile Software Development.